

# Analysis: Cryptopangrams

### Yeah, but what about *decrypting*?

The statement tells us how the plaintext is encrypted, but it says nothing about the decryption mechanism! Figuring that out is part of the problem. Since this is a Qualification Round problem, there is slightly less time pressure and competitive urgency, and you have some extra time to think about how this cryptosystem is supposed to be used.

Suppose that Cameron and Jamie are members of the Code Jam team who know the secret list of 26 primes, and suppose that Cameron has just sent the ciphertext to Jamie. Each value in the ciphertext is the product of two of those primes, so Jamie can try dividing each value by each known prime to find the value's factors. (Notice that a value in the ciphertext might be the square of a prime, if the plaintext has the same letter twice in a row.)

After getting the list of factored ciphertext values, how should Jamie recover the plaintext? We might be tempted to say that the second letter of the plaintext is the one whose prime appears as factors of both the first and second ciphertext values, the third letter is the one whose prime appears as factors of both the second and third ciphertext values, and so on. Then the remaining factors in the first and last ciphertext values give us the first and last letters of the plaintext.

This is almost correct, but we (and Jamie) would have to deal with one significant annoyance. If the plaintext starts with something like `ABABA...`, for example, then the first, second, third, and fourth ciphertext values will all be the same, being the product of the same two factors (the primes corresponding to `A` and `B`). In particular, the start of `BABAB...` looks just the same as the start of `ABABA...`! The good news is that we know that this kind of pattern must terminate somewhere in the message; eventually, either we will get the same letter twice in a row, or (since the plaintext uses more than two different letters) three consecutive different letters. As soon as either of these things happens, we have a "break-in point", and we know which factors of a particular ciphertext value go with which of the two letters of the plaintext that it encodes. Then, we can "unzip" the rest of the plaintext from there, working backwards and forwards.

For instance, if the plaintext starts with `ABABAABAB`, the first four ciphertext values will all be the same: the product of the prime corresponding to `A` and the prime corresponding to `B`. The fifth ciphertext value will represent the square of `A`, so we will know that the fifth and sixth plaintext letters are both `A`. We can then reason that the fourth plaintext letter must be the fourth ciphertext value divided by the prime corresponding to `A`, the third plaintext letter must be the third ciphertext value divided by the prime corresponding to `B`, and so on going backwards. We can also determine that the seventh plaintext letter is the sixth ciphertext value divided by the prime corresponding to `A`, and so on going forwards.

Similarly, if the plaintext starts with `ABABCBABA`, when we inspect the third and fourth ciphertext values, we will see that they are different, but both have the prime corresponding to `B` as a factor. Then we can unzip from there, as above.

However, we must remember that Jamie has an advantage that we do not have: we do not know the 26 secret primes! We need to find a way to get them.

### Test set 1

In test set 1, the ciphertext values are products of small primes. Each prime is less than  $10^4$ , so each ciphertext value is no larger than  $10^8$ . It is straightforward to factor these values by testing every possible (prime) factor between 2 and  $10^4$ . Once we have all of the factors, we will have our set of 26 primes, since each prime will be represented in at least one factor of one ciphertext value. We can assign them in increasing order to the letters A through Z.

Then, to recover the plaintext, we can use a bit of brute force to sidestep the need to unzip as described before. Consider the two factors that contribute to the first ciphertext value; arbitrarily choose one of them. Let us first assume that that factor corresponds to the first letter of the plaintext, and the other corresponds to the second. Then we can take the remaining factor and try to divide the second ciphertext value by that factor. If we cannot, we have a contradiction, and we should go back and make the other factor correspond to the first letter of the plaintext. Otherwise, the quotient is the factor corresponding to the third letter of the plaintext, and so on. For one of our choices, this method will work and will give us the correct plaintext; for the other choice, we will reach a contradiction, since (as described above) it is guaranteed that there is only one possible decryption.

## Test set 2

In test set 2, the primes can be enormous (as large as a googol), and the product of two such primes can be even more enormous. We should realize that it is hopeless to try to factor such a product. If we could do that, we could also break modern cryptosystems that depend on the assumption that factoring large numbers is intractable! The Code Jam servers do not run on quantum computers (yet...), so there is no way for us to try to use a quantum algorithm, either.

To solve this seemingly impossible test set, we need to find a different vulnerability of this cryptosystem. The key insight is that any two consecutive values in the ciphertext have at least one factor in common. Factoring very large numbers may be intractable, but we *do* know how to efficiently find the greatest common divisor of two very large numbers! A method like [Euclid's algorithm](#) will be [fast enough](#).

Notice that if we have a plaintext like ABABC..., it is possible that the prime corresponding to A will not appear in any of the pairwise GCD values. So, we should compute GCDs of consecutive ciphertext values until we get a value that is not 1; at least one such value must exist, as described in the introduction to the problem. At that point, we can unzip the rest of the ciphertext, as described previously, finding all of the primes as we go. Then we can proceed as above. And we do not even have to know a bevy of DP flux algorithms, whatever those are!

## A note on language choice

An essential skill in Code Jam is picking the right tool (language) for the right job. For some problems, a fast language such as C++ may be needed to obtain a solution within the time limits. For this problem, it is probably better to choose a language like Python that can handle very large numbers without any extra hassle, even at the cost of some speed, or a language like Java that has a built-in "big integer" implementation.

# Analysis: Dat Bae

## Test set 1

In this problem, we need to somehow identify which workers are not returning the bits that we send to them. Let's see how strings of bits change when some of the data is lost.

Imagine that we have ten workers, and we send them the following five random strings of bits (the  $i$ -th bit in each string is sent to the  $i$ -th worker):

```
0101010110
0101010101
0010100100
0110110101
0100100100
```

Also, imagine that workers 3 and 6 are broken. In this case, we will lose the following bits (highlighted in bold):

```
0101010110
0101010101
0010100100
0110110101
0100100100
```

In the result, we will receive the following bit strings:

```
01001110
01001101
00110100
01111101
01010100
```

Notice, how in this representation, we lost *columns* of bits because of the broken workers. It would be nice to be able to tell which columns we lost – then we will be able to determine which workers did not return the data!

Let's see if we can make all the columns different – then it will be easy to tell which ones are missing. In test set 1 we have up to  $N = 1024$  workers, so we will need up to 1024 different columns. We can send up to  $F = 10$  bit strings, which means our columns will consist of up to 10 bits.

Fortunately for us,  $2^{10} = 1024$ , so we can make each column represent a unique number in the range from 0 to 1023. For example, we could make the  $i$ -th column represent the number  $i$ , in which case the first five columns, that represent numbers from 0 to 4, could look like this:

```
01010...
00110...
00001...
00000...
00000...
00000...
00000...
```

00000...  
00000...  
00000...

Now we can use a construction like this to see which workers are broken. If the  $i$ -th worker is broken, the bit column representing the number  $i$  will be missing in the result we receive.

## Test set 2

In the second test set, we can only send up to  $F = 5$  bit strings, which means our columns will consist of only up to 5 bits.  $2^5 = 32$ , so we can no longer use a previous approach of making each column unique.

Notice how we also know that  $B \leq \min(15, N-1)$ , even though we have not used this fact in our solution so far. How does this additional constraint change the problem? The first thing to notice is that only a small fraction of columns will be missing when  $N = 1024$ , but these columns can still be in any positions.

Let's see what we can do with 32 numbers that we can represent with 5 bits. If we put these numbers in an order, that is

0, 1, ..., 31

we can notice that since  $B$ , which is less than 15, is also less than 32, this whole block of 32 numbers will never disappear completely. Let's see how we can make further use of this fact. If we have several blocks like this:

0, 1, ..., 31, 0, 1, ..., 31, 0, 1, ..., 31, ...

none of the blocks of numbers from 0 to 31 will disappear completely, and for each the remaining numbers, we will always be able to identify which block it is from.

Let's examine this in more detail. Numbers inside each block are in an increasing order. Notice that even after some numbers disappear, when one block ends and the next one starts, numbers go down:

0, 1, ..., 31, 0, 1, ..., 27, 5, ..., 31, ... (numbers between 27 and 5 disappeared, but 27 is still bigger than 5)

Assume this is not the case, and after some number  $x$  from one block goes a number  $y$  from the next block such that  $x \leq y$ . But this is impossible since there were at least 31 numbers between any such  $x$  and  $y$ , and these numbers could not disappear altogether.

With these observations at hand, we are ready for a final solution. Let the bit columns of the strings we send to the database represent repeating blocks of numbers from 0 to 31:

0, 1, ..., 31, 0, 1, ..., 31, 0, 1, ..., 31, ... ( $N$  numbers total)

After we receive the responses from the database, we can iterate through the remaining numbers, noting that the new block starts when the current number is smaller than the previous one, and keeping track of how many blocks we have seen so far. Knowing the position of the numbers inside the block and the number of blocks we have seen, we can uniquely identify all the numbers we see: for example number 16 in the fifth block is  $(5 - 1) * 32 + 17 = 145$ -th number in the whole sequence. And if we know which numbers we have seen in the whole sequence, we can find out which numbers are missing, and output them as the numbers of the missing workers.

Finally, we used the fact that  $\mathbf{B} < 2^5$  to make the approach above work. But  $\mathbf{B} < 2^4$  too! That means we can use the same approach with only four strings. In this case, it is possible that two consecutive numbers from different blocks are equal, since there are  $2^4 - 1 = 15$  other numbers in between them, which might all get deleted. Therefore, to detect block changes, we must use  $\geq$  instead of  $>$  between consecutive numbers.

# Analysis: Foregone Solution

There are at least three different approaches to this problem, and they will have different degrees of success!

## Brute force

In test set 1,  $N$  is less than 100000, so if we want to find two numbers  $A$  and  $B$  such that  $A + B = N$ , there are fewer than 100000 possible choices for  $A$ . (Once we choose a candidate value for  $A$ , we know that  $B = N - A$ .) We can try all possible choices of  $A$  until we find one for which neither  $A$  nor  $B$  contains a 4. However, we cannot apply this strategy to test set 2, in which we might have to check almost  $10^9$  values!

## Randomized

We can repeatedly choose a candidate value of  $A$  uniformly at random from the inclusive range  $[1, N-1]$ , and check whether both it and  $B = N - A$  are free of the dreaded 4s. But will this randomized approach find a solution fast enough?

How many numbers less than  $10^9$  do not contain a 4? Consider the set of all 9-digit numbers, with leading zeroes allowed; notice that this set therefore includes all 8-digit, etc. numbers. If we choose a number at random from this set, the probability that it lacks a 4 is the probability that its first digit is not a 4, times the probability that its second digit is not a 4, and so on. Since the probability is  $9/10$  for any one digit, and the probabilities for different digits are independent, we have an overall probability of  $(9/10)^9$ , which is about 0.387. In practice, depending on the actual value of  $N$ , we might have fewer valid choices, but the size of this answer should give us some hope that we can find our  $A$  and  $B$  quickly. If we presume that the probability that  $A$  has no 4s is not too much smaller than our estimate, and that  $A$  not having a 4 does not substantially increase the odds that  $B$  has a 4, there should be many working values of  $A$  out there to find.

We can argue more rigorously. Let  $D$  be the number of digits in  $N$ ; we will consider the set of all  $D$ -digit numbers less than  $N$ , with leading zeroes allowed. Suppose that we choose  $A$  randomly from that set, and let  $B = N - A$ , padding with leading zeroes as needed to bring  $A$  and  $B$  up to  $D$  digits.

Consider the last digit of  $A$ . If it is a 4, or if it is  $(N - 4) \bmod 10$  (which means that the last digit of  $B$  is a 4), then we have failed. But at least 8 of the 10 possible values for the last digit are OK. Then, assuming we have not yet failed, consider the next-to-last digit of  $A$ . There is at most one value for that digit that would imply that the next-to-last digit of  $B$  is a 4, so at least 8 of the 10 possible values for the next-to-last digit of  $A$  are OK. Notice that even though the value of the last digit of  $A$  might determine which *particular* value for the next-to-last digit of  $A$  would imply that the next-to-last digit of  $B$  is a 4, we can still be sure that there exist at least 8 OK values.

And so on, for all  $D-1$  digits up to the first. We only need to worry about the first digit of  $A$  or  $B$  being a 4 if the first digit of  $N$  is at least 4, in which case there are certainly at least five possible values for the first digit of  $A$ , of which (just as before) at most two can be bad. So the overall probability that a randomly chosen  $A$  is OK is no smaller than  $(8/10)^{D-1} \times (3/5)$ . For  $D = 9$ , this is around 0.1, so our randomized approach should only need an expected 10 or so tries to find an answer, and the probability that it will fail to find one after e.g. 1000 tries is vanishingly small (around  $10^{-46}$ ).

If you check all values with  $D$  digits for some small  $D$ , you will find that the worst case (i.e. the one with the fewest solutions) is a 4 followed by  $D-1$  9s. For  $N = 49999999$ , the fraction of possible choices that will work is around 0.1258, so our bound of 0.1 was not particularly conservative!

However, this approach is highly unlikely to work for test set 3, in which  $N$  can be on the order of  $10^{100}$ . Our lower bound on the probability that a randomly chosen solution is correct becomes  $(8/10)^{99} \times (3/5)$ , which is about  $1.5 \times 10^{-10}$ . Even if we try a more clever randomized approach in which we build a random  $A$  (less than  $N$ ) out of non-4 digits and then check the corresponding  $B$ , which would change those  $8/10$  factors to  $9/10$ , that bound would be on the order of  $10^{-5}$ , which seems too low to work well on 100 test cases. Can we find a deterministic solution?

## Constructive

Observe that we can write  $N$  in terms of a sum of powers of 10 times its digits. For example,  $4837 = 4 \times 1000 + 8 \times 100 + 3 \times 10 + 7 \times 1$ .

Notice that every digit can be written as a sum of two non-4 digits. In particular, we can write  $4 = 2 + 2$ , and for any other digit  $X$ , we can write  $X = 0 + X$ .

We can apply these observations to solve our problem digit by digit! In the above example, we can decompose  $4 \times 1000$  into  $2 \times 1000$ ; and  $2 \times 1000$ ;  $8 \times 100$  into  $0 \times 100$  and  $8 \times 100$ , and so on. In this way, we are building up the digits of our desired  $A$  and  $B$ ;  $A$  is  $2 \times 1000 + 0 \times 100 \dots = 2000$ , and  $B$  is  $2 \times 1000 + 8 \times 100 \dots = 2837$ .

In summary, we can set  $A$  to be the same as  $N$  with every 4 replaced with a 2, and every non-4 replaced with a 0.  $B$  is simply  $N$  with every 4 replaced with a 2. Even though we would normally need a "big integer" implementation to handle numbers as large as a googol, in this case we can implement the solution using only strings!

# Analysis: You Can Go Your Own Way

### Test set 1

In the first test set, we can construct all possible paths from the northwest cell of the maze to the southeast cell using [backtracking](#), and see which of them satisfy our requirements (we know from the Output section that at least one answer exists). When we find one that works, we output it as an answer for the test and stop looking any further.

### Test set 2

Let's assess the number of possible paths from the northwest cell to the southeast cell. For a maze of size  $N$  by  $N$ , every possible valid path makes  $N - 1$  moves to the east and  $N - 1$  moves to the south. Notice that the order in which these moves are made does not matter – we will always arrive at the southeast cell after making all of them and we are guaranteed not to leave the maze in the process.

So we need to make  $2N - 2$  moves total, out of which  $N - 1$  are to the east and  $N - 1$  are to the south, and the order does not matter. Using [combinations](#), we can see that there are  $\binom{2N - 2}{N - 1}$  possible options. For  $N \leq 10$  in the test set 1 there are at most 48620 available paths that we need to check. For test set 2, however, in which  $N = 100$ , there are

22750883079422934966181954039568885395604168260154104734000 (or

approximately  $2.28 \times 10^{58}$ ) possible paths to take. This is too much to process in the time limit, so let's think of an alternative solution.

We can think of a maze as a [graph](#), where the unit cells are nodes and there is an edge between every pair of nodes that represent neighboring cells. Now instead of moving between two neighboring cells, we will move between the corresponding nodes along the edge connecting them. Because we cannot reuse Lydia's moves, the edges that she used before are no longer available to us, and we remove them from the graph. After that, our problem of finding a valid path reduces to the problem of finding *any* path from the node representing the northwest cell to the node representing the southeast cell. This is a standard graph problem that can be solved using either [Depth-first search](#) or [Breadth-first search](#) in  $O(N^2)$  time, which is fast enough to pass this test set.

### Test set 3

With  $N \leq 50000$  now, we must think of a different approach to the problem.

To solve this test set, let's just invert all of the moves in Lydia's path. That is, every time she moves east, let's move south, and every time she moves south, let's move east. For example, if Lydia's path is EESSSESE, then our path will be SSEEESES.

Let's understand why this inverted path is a correct answer to the problem.

First, notice that we still make  $N - 1$  moves to the east and  $N - 1$  moves to the south, so we will arrive at the southeast cell in the end as required, and we will not step out of the bounds of the maze.

Now let's see why we will not reuse any of Lydia's moves. Suppose this is not the case, and we reuse a move from the position that is  $x$  moves to the east and  $y$  moves to the south in some



order from the northwest cell. Recall that the order of moves does not matter, and there may be many ways to get to this position, but all of them will require exactly  $x$  moves to the east and  $y$  moves to the south. What will be the next move? We know that Lydia's next move is the  $(x + y + 1)$ -th symbol in the string representing her path (with indexing starting from one), and our next move is the  $(x + y + 1)$ -th symbol in the string representing our path. But since our path string is an inverted version of Lydia's path string, we know that  $(x + y + 1)$ -th symbols of the two strings will be different, which contradicts our assumption that we will have the same next move. By the same logic, we can see that the two paths will not reuse any other moves along the way.